
CHAPTER NO 7.
EXCEPTION HANDLING

- INTRODUCTION,
 - EXCEPTION HANDLING MECHANISM,
 - CONCEPT OF THROW AND CATCH WITH EXAMPLE
-

INTRODUCTION TO EXCEPTION HANDLING.

- An exception is a problem that arises during the execution of a program.
- A C++ exception is a response to an exceptional circumstance that arises while a program is running , such as an attempt to divide by zero.
- Exceptions provide a way to transfer control from one part of a program to another.
- C++ exception handling is built upon three keywords: try, catch, and throw.
 - **throw**: A program throws an exception when a problem shows up. This is done using a **throw** keyword.
 - **catch**: A program catches an exception with an exception handler at the place in a program where you want to handle the problem. The catch keyword indicates the catching of an exception.
 - **try**: A try block identifies a block of code for which particular exceptions will be activated. It's followed by one or more catch blocks.
- Assuming a block will raise an exception, a method catches an exception using a combination of the try and catch keywords.
- A try/catch block is placed around the code that might generate an exception.
- Code within a try/catch block is referred to as protected code.
- **Syntax for using try/catch is as follow.**

```
try
{
    // protected code
} catch( ExceptionName e1 )
{
    // catch block
} catch( ExceptionName e2 )
{
    // catch block
} catch( ExceptionName eN )
{
    // catch block
}
```

- Sometimes your try block raises more than one exception in different situation then you can list down multiple catch statements to catch different type of exception.

THROWING EXCEPTIONS:

- Exceptions can be thrown anywhere within a code block using throw statements.
- The parameter of the throw statements determines a data type for the exception.
 - It can be any expression e.g. throw **10** (int) or throw "**Exception Error**" (char *)
 - The type of the result of the expression determines the type of exception thrown.
- **Example :** Following function throwing an exception when dividing by zero condition occurs:

```
double division(int a, int b)
{
    if( b == 0 )
    {
        throw "Division by zero condition!";
    }
    return (a/b);
}
```

- You must have to call this function from the try block and handle the exception in catch block because this function may throw the exception.

CATCHING EXCEPTIONS:

- The catch block following the try block catches any exception.
- You can specify what type of exception you want to catch and this is determined by the exception declaration that appears in parentheses following the keyword catch.

- **Syntax 1:** When Exception type is mention

```
try
{
    // protected code
}catch( ExceptionName e )
{
    // code to handle ExceptionName exception
}
```

- Above code will catch an exception of **ExceptionName** type.
- If you want to specify that a catch block should handle any type of exception that is thrown in a try block, you must put an ellipsis, ..., between the parentheses enclosing the exception.
- **Syntax 2:** When Exception type is not mention

```
try
{
    // protected code
}catch( ... )
{
    // code to handle any exception
}
```

EXAMPLE OF EXCEPTION HANDLING.

Write a program which throws a **division by zero exception** and we catch it in catch block.

[Hint : When we divide any number by zero it produce run time error divide by zero and program terminates abnormally.]

```
#include <iostream>
using namespace std;
double division(int a, int b)
{
    if( b == 0 )
    {
        throw "Division by zero condition!"; //Type of Exception is String (char *)
    }
    return (a/b);
}

int main ()
{
    int x = 50;
    int y = 0;
    double z = 0;
    try
    {
        z = division(x, y);
        cout << z << endl;
    }catch (const char* msg)
    {
        cerr << msg << endl;
    }
    return 0;
}
```

- Observe that, we are raising (throw) an exception of type const char*, so while catching this exception, we have to use const char* in catch block.

DEFINE NEW EXCEPTIONS (USER DEFINE) : (Not in your syllabus)

- You can define your own exceptions by inheriting and overriding exception class functionality.
- Following is the example, which shows how you can use std::exception class to implement your own exception in standard way:

<pre>#include <iostream> #include <exception> using namespace std; struct MyException : public exception { const char * what () const throw ()</pre>	<pre>int main() { try { throw MyException(); }</pre>
--	--

```
{  
    return "C++ Exception";  
}  
};
```

```
catch(MyException & e)  
{  
    std::cout << "MyException caught" <<  
        std::endl;  
    std::cout << e.what() << std::endl;  
}  
catch(std::exception & e)  
{  
    //Other errors  
}  
}
```

- Here, what() is a public method provided by exception class and it has been overridden by all the child exception classes. This returns the cause of an exception.

GALAXY
COMPUTER EDUCATION

Our Talent Spins The World ...